

Taller de Matemáticas Discretas

Documento de explicación — Seguridad, Redes, Lógica y Computación
Cuántica

Erik Santiago Martinez Perez

oday

Tabla de contenido

# Documento de explicación — Taller de Matemáticas Discretas	1
Ejercicio 1 — Cifrado César	1
Ejercicio 2 — RSA de juguete	2
Ejercicio 3 — MPC básico: promedio sin mostrar los datos	3
Ejercicio 4 — Ruta más corta (Dijkstra)	4
Ejercicio 5 — Cierre de una estación: impacto en la red	5
Ejercicio 6 — Coloreo de grafos: organizar exámenes sin choques	6
Ejercicio 7 — Tablas de verdad y circuitos lógicos	7
Ejercicio 8 — Simplificación booleana: hacer un circuito más barato	8
Ejercicio 9 — Shannon: medir información en un mensaje	9
Ejercicio 10 — Primer simulador cuántico: bits, qubits y mediciones	11

Documento de explicación — Taller de Matemáticas Discretas

Ejercicio 1 — Cifrado César

Categoría: cripto

1. ¿Qué problema resuelve el programa?

Cifra y descifra mensajes con el cifrado César, y además rompe un mensaje cifrado cuando no se conoce el desplazamiento, probando las 26 posibilidades.

2. ¿Qué idea matemática usa?

Aritmética modular sobre $\mathbb{Z}_{26}$. El cifrado es la función afín $f(x) = (x+k) \bmod 26$. Como esa suma es invertible (restar k , o sumar $26-k$), el cifrado se deshace aplicando el desplazamiento contrario. La fuerza bruta es posible porque el espacio de llaves es de solo 26 valores.

3. ¿Cómo se ejecuta?

python src/Bloque_A-Criptografía/ejercicio_01.py

Al correrlo, primero muestra el ejemplo del enunciado y la fuerza bruta de un segundo texto. Luego entra en modo interactivo y pregunta: (1) Cifrar, (2) Descifrar o (3) Fuerza

bruta — pidiendo el texto (y k, si aplica) por teclado.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
cifrar("HOLA UNAL", 3)	"KROD XQDO"	No (ejemplo del enunciado)
descifrar(cifrar(texto,k),k) para k=1,5,13,25	texto original	Sí (k grande)
cifrar("Sala #5, 22:00!", 7)	signos/números sin cambio	Sí (no alfabéticos)
cifrar(texto,0) y cifrar(texto,26)	igual al original	Sí (desplazamiento nulo/vuelta completa)
fuerza_bruta(cifrado)	incluye el texto original entre 26 opciones	Sí (llave desconocida)

5. ¿Qué limitaciones tiene la solución?

Solo funciona con A-Z sin ñ. Tildes, ñ y símbolos de otros idiomas quedan sin cifrar. No detecta automáticamente cuál de los 26 resultados es el correcto; eso lo decide quien lee la salida.

Ejercicio 2 — RSA de juguete

Categoría: cripto

1. ¿Qué problema resuelve el programa?

Genera llaves RSA (n, d) a partir de dos primos p, q y un exponente público e, y usa esas llaves para cifrar y descifrar un mensaje numérico M. Es una versión educativa: no debe usarse como seguridad real.

2. ¿Qué idea matemática usa?

Aritmética modular y el teorema de Euler. $n = pq$, $\phi(n) = (p-1)(q-1)$. El exponente privado d es el inverso modular de e módulo $\phi(n)$ (calculado con el algoritmo de Euclides extendido), que existe solo si $\gcd(e, \phi(n)) = 1$. Cifrado y descifrado son congruencias: $C \equiv M^e \pmod{n}$, $M \equiv C^d \pmod{n}$. El teorema de Euler garantiza que $M^{ed} \equiv M \pmod{n}$ porque $e*d \equiv 1 \pmod{\phi(n)}$. Los primos p, q son el secreto: sin ellos no se puede calcular $\phi(n)$ ni d.

3. ¿Cómo se ejecuta?

python src/Bloque_A-Criptografía/ejercicio_02.py

Muestra el caso obligatorio del enunciado y un segundo ejemplo con otros números. Luego entra en modo interactivo y pide p, q, e y M por teclado.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
p=61,q=53,e=17,M=65	n=3233, phi=3120, d=2753, C=2790, M=65	No (caso obligatorio)
p=17,q=11,e=7,M=88	C=11, descifrado=88	No (otro ejemplo conocido)
(17*d) mod phi(n)	1	No (verifica inverso modular)
p=5,q=11,e=8	lanza ValueError (gcd(e,phi)=8≠1)	Sí (e inválido)
p=5,q=11,e=3,M=55 (M=n)	descifrado=0	Sí (M no menor que n)

5. ¿Qué limitaciones tiene la solución?

Solo funciona correctamente si $M < n$; si $M \geq n$, el resultado se reduce módulo n y se pierde información (limitación matemática de RSA, no un bug). No verifica que p y q sean primos ni que e sea válido en rango; solo valida que $\gcd(e, \phi(n)) = 1$. Con primos pequeños no ofrece seguridad real.

Ejercicio 3 — MPC básico: promedio sin mostrar los datos

Categoría: cripto

1. ¿Qué problema resuelve el programa?

Simula un cálculo multipartito seguro (MPC): tres servidores calculan la suma y el promedio de una lista de notas sin que ninguno vea una nota completa en ningún momento.

2. ¿Qué idea matemática usa?

Secret sharing aditivo módulo M . Cada nota x se reparte en 3 partes aleatorias s_1, s_2, s_3 tales que $x \equiv s_1 + s_2 + s_3 \pmod{M}$. Ejemplo pequeño con $M=97$ y $x=40$: se eligen al azar $s_1=53$ y $s_2=61$, y se calcula $s_3 = (40 - 53 - 61) \bmod 97 = 23$. Comprobación: $(53+61+23) \bmod 97 = 137 \bmod 97 = 40$. Cada s_i por separado es un número “al azar” en $[0, M)$ que no revela nada de x ; solo sumando las 3 partes se recupera x . Cada servidor suma únicamente las partes que le tocaron de todas las notas (suma parcial); al sumar las 3 sumas parciales (mod M) se reconstruye la suma total real, sin que ningún servidor haya reconstruido una nota individual.

3. ¿Cómo se ejecuta?

python src/Bloque_A-Criptografía/ejercicio_03.py

Corre el ejemplo del enunciado y un segundo ejemplo con más notas. Luego pide en modo interactivo una lista de notas separadas por coma.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
[40, 35, 50, 25]	suma=150, promedio=37.5	No (ejemplo del enunciado)
[10]	suma=10, promedio=10.0	Sí (una sola nota)
lista de 50 notas	suma y promedio coinciden con cálculo directo	Sí (lista grande)
vistas de cada servidor	ninguna coincide con la lista real de notas	No (verifica privacidad)
[] (lista vacía)	lanza ValueError	Sí (lista vacía)
[40, 60, 10] (60 fuera de rango)	lanza ValueError	Sí (nota inválida)

5. ¿Qué limitaciones tiene la solución?

Es una simulación en un solo programa: los “servidores” son listas en memoria, no procesos ni máquinas separadas realmente aisladas. Solo funciona bien si la suma real de las notas es menor que M; con M=1000003 y notas de 0 a 50, esto es seguro para cursos de tamaño normal, pero no se valida automáticamente si M fuera más pequeño o el curso enorme.

Ejercicio 4 — Ruta más corta (Dijkstra)

Categoría: grafos

1. ¿Qué problema resuelve el programa?

Encuentra la distancia mínima y la ruta más corta entre dos puntos de una ciudad, representada como un grafo ponderado (vértices = lugares, aristas = conexiones con un tiempo/distancia).

2. ¿Qué idea matemática usa?

El algoritmo de Dijkstra, basado en la propiedad de subestructura óptima: si el camino más corto de A a C pasa por B, el tramo A->B también es el camino más corto entre A y B. Usando una cola de prioridad, siempre se “cierra” el vértice no confirmado con menor distancia tentativa. Requiere pesos no negativos porque, si un peso fuera negativo, un vértice ya cerrado podría mejorarse después, y el algoritmo no lo detectaría (para esos casos se necesita Bellman-Ford). Un camino óptimo es aquel cuya suma de pesos es la mínima posible entre todos los caminos que unen ese par de vértices.

3. ¿Cómo se ejecuta?

python src/Bloque_B-Grafos/ejercicio_04.py

Usa el grafo de prueba GRAFO_CIUDDAD (8 vértices, 13 aristas) definido en el código. También se puede cargar el mismo grafo desde tests/grafos/grafos_ejemplo_ej4.json con

`cargar_grafo_desde_archivo(...)`. Al correrlo, muestra dos ejemplos y luego entra en modo interactivo pidiendo nodo origen y destino.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
Portal -> Estadio	distancia=17, ruta Portal-Calle26-Museo- Centro-Universidad- Parque-Estadio	No
Portal -> Terminal	distancia=8 (arista directa, mejor que cualquier rodeo)	No
Centro -> Centro	distancia=0, ruta=[Centro]	Sí (origen=destino)
Portal -> Aeropuerto (no existe)	lanza ValueError	Sí (nodo inexistente)
grafo con nodo aislado Z	distancia=None, ruta=[]	Sí (nodo inalcanzable)
grafo con peso -3	lanza ValueError al cargar	Sí (peso negativo)

5. ¿Qué limitaciones tiene la solución?

Solo funciona correctamente con pesos no negativos (validado al cargar el grafo). No maneja grafos dirigidos con reglas de sentido único de forma especial (si se quiere un grafo dirigido, simplemente no se agrega la arista de vuelta). No calcula todas las rutas más cortas simultáneamente (solo una consulta origen-destino a la vez).

Ejercicio 5 — Cierre de una estación: impacto en la red

Categoría: grafos

1. ¿Qué problema resuelve el programa?

Mide qué tan grave es cerrar un punto de la red de transporte: compara las distancias más cortas entre varios pares de lugares antes y después de eliminar un vértice (o una arista), y reporta cuáles rutas se alargaron y cuáles quedaron sin camino posible.

2. ¿Qué idea matemática usa?

Reutiliza Dijkstra sobre dos versiones del mismo grafo: G (original) y $G' = G$ sin el vértice/arista cerrado. Para cada par se compara $d_G(\text{origen}, \text{destino})$ contra $d_{G'}(\text{origen}, \text{destino})$. Esto se relaciona con la noción de “punto de corte” (cut vertex) o “puente” (bridge): un vértice/arista cuya eliminación aumenta el número de componentes conexas, es decir, desconecta pares que antes sí tenían camino.

3. ¿Cómo se ejecuta?

python src/Bloque_B-Grafos/ejercicio_05.py

Se usa el grafo del ejercicio 4, agregando un nodo Biblioteca que solo se conecta por Centro, para poder mostrar un caso real de desconexión (no solo rutas más largas). Se cierra el vértice Centro y se comparan 5 pares de nodos; luego se muestra un segundo ejemplo cerrando solo la arista Universidad-Parque. Al final hay un modo interactivo para elegir qué cerrar y con qué pares.

4. ¿Qué pruebas se hicieron?

Par	Antes	Después	Estado
Portal → Estadio	17	19	ruta más larga
Portal → Museo	7	7	sin cambio
Calle26 → Parque	9	11	ruta más larga
Museo → Terminal	15	15	sin cambio
Portal → Biblioteca	11	—	quedó desconectado
Portal → Centro (el nodo cerrado)	9	—	estación cerrada (nodo eliminado)

También se probó que cerrar solo una arista (no un vértice completo) no afecta el resto del grafo, y que intentar cerrar un vértice inexistente lanza un error claro.

5. ¿Qué limitaciones tiene la solución?

Solo evalúa el impacto sobre los pares que se le indiquen explícitamente, no calcula automáticamente todos los pares posibles de la red. El grafo usado es denso (pocos puntos de corte reales); para ver una desconexión “natural” entre dos nodos que no sea el propio nodo cerrado, se agregó Biblioteca como nodo de ejemplo con una sola conexión.

Ejercicio 6 — Coloreo de grafos: organizar exámenes sin choques

Categoría: grafos

1. ¿Qué problema resuelve el programa?

Asigna franjas horarias (colores) a cursos de forma que dos cursos con estudiantes en común nunca queden en la misma franja, usando un grafo de conflictos donde cada arista indica que dos cursos comparten estudiantes.

2. ¿Qué idea matemática usa?

Coloreo propio de vértices: una asignación de colores donde dos vértices adyacentes nunca comparten color. El algoritmo voraz recorre los vértices en un orden dado (aquí, por grado descendente — heurística de Welsh-Powell: primero los cursos con más conflictos) y a cada uno le asigna el color más pequeño que ninguno de sus vecinos ya coloreados esté usando. Esto siempre da un coloreo válido, pero no necesariamente el mínimo número de colores posible (el “número cromático” del grafo), porque decide vértice por vértice sin ver el grafo completo.

3. ¿Cómo se ejecuta?

python src/Bloque_B-Grafos/ejercicio_06.py

Usa el grafo GRAFO_CURSOS (10 cursos, 12 conflictos) definido en el código. Muestra un segundo ejemplo con un grafo bipartito donde se comparan dos órdenes de procesamiento distintos. Al final hay un modo interactivo para ingresar un grafo de conflictos propio.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
GRAFO_CURSOS (10 cursos, 12 aristas)	coloreo válido con 3 colores	No
Grafo completo K4	coloreo válido con exactamente 4 colores	Sí (cada vértice vecino de todos)
Grafo bipartito, orden "malo"	coloreo válido pero con 3 colores (subóptimo)	Sí (demuestra el punto 5)
Mismo grafo bipartito, orden por grado	coloreo válido con 2 colores (óptimo)	Sí (mismo grafo, mejor orden)
Grafo sin aristas	coloreo válido con 1 solo color	Sí (sin conflictos)
Colores forzados iguales en un par adyacente	verificar_coloreo detecta la falla (False)	Sí (verifica el validador)

5. ¿Qué limitaciones tiene la solución?

El algoritmo voraz no garantiza el número mínimo de colores (número cromático): como se ve en la prueba del grafo bipartito, el mismo grafo puede necesitar 2 o 3 colores según el orden de procesamiento. Encontrar el número cromático exacto es un problema NP-difícil en general, por lo que aquí se usa una heurística (orden por grado descendente) que en la práctica da buenos resultados, pero no los óptimos garantizados.

Ejercicio 7 — Tablas de verdad y circuitos lógicos

Categoría: boole

1. ¿Qué problema resuelve el programa?

Genera la tabla de verdad de una expresión booleana (variables A,B,C,D con AND, OR, NOT, XOR) y permite evaluarla en una entrada concreta.

2. ¿Qué idea matemática usa?

Un evaluador propio: tokenizador + parser de descenso recursivo que respeta precedencia (NOT > AND > XOR > OR, y los paréntesis mandan), construyendo un árbol de sintaxis que se recorre para evaluar. La tabla de verdad es la enumeración exhaustiva de las 2^n combinaciones de entrada de esa función booleana. Cada

conectivo corresponde a una compuerta lógica física (AND, OR, NOT/inversor, XOR), así que la expresión completa equivale a una red de compuertas, y la tabla de verdad es la especificación que ese circuito debe cumplir para cada estado posible de sus entradas.

3. ¿Cómo se ejecuta?

python src/Bloque_C-Boole-Shannon-ComputaciónCuántica/ejercicio_07.py

Imprime las tablas de las 3 expresiones obligatorias, muestra que la misma expresión con símbolos ($\wedge \vee \neg$) da el mismo resultado que con palabras, y evalúa un par de entradas concretas. Al final hay un modo interactivo para ingresar cualquier expresión propia.

4. ¿Qué pruebas se hicieron?

Expresión / entrada	Resultado esperado	¿Caso límite?
(A AND B) OR (NOT C), fila A=V,B=V,C=F	V	No
(A AND B) OR (NOT C), fila A=F,B=F,C=V	F	No
(A XOR B) AND C, fila A=V,B=F,C=V	V	No
(A OR B) AND (NOT A OR C), fila A=V,B=F,C=F	F	No
evaluar_expresion con entrada concreta	coincide con la fila de la tabla	No
misma expresión con símbolos $\wedge \vee \neg$	tabla idéntica a la de palabras	Sí (equivalencia de notación)
“(A AND B)” (paréntesis sin cerrar)	lanza ValueError	Sí (expresión mal formada)

5. ¿Qué limitaciones tiene la solución?

Los caracteres no reconocidos (letras fuera de A-D, comas, etc.) se ignoran silenciosamente en vez de marcar error, así que una expresión con un typo puede parsear “algo” en vez de avisar claramente. No simplifica la expresión (no reduce a SOP/POS mínimos), solo la evalúa tal cual.

Ejercicio 8 — Simplificación booleana: hacer un circuito más barato

Categoría: boole

1. ¿Qué problema resuelve el programa?

Recibe los minterminos de una función booleana de 3 o 4 variables y produce una expresión simplificada en suma de productos, con menos compuertas que la suma directa de minterminos, sin cambiar el comportamiento de la función.

2. ¿Qué idea matemática usa?

Un mintermino es un término producto donde aparecen TODAS las variables (complementadas o no), correspondiente a una sola fila de la tabla de verdad donde la función vale 1; la suma de todos los minterminos es, por construcción, una expresión con la misma tabla de verdad que la función. Quine-McCluskey combina repetidamente términos que difieren en un solo bit (ley de combinación $X \cdot Y + X \cdot \neg Y = X$) hasta obtener los “implicantes primos”, y luego selecciona el menor número de ellos que cubra todos los minterminos (esenciales primero, el resto con una heurística voraz). Dos expresiones son equivalentes si y solo si tienen la misma tabla de verdad: eso es lo que se compara literalmente, fila por fila, para confirmar que la simplificación no alteró el circuito. No se usó ninguna librería externa; tanto la simplificación como la verificación están implementadas en este archivo.

3. ¿Cómo se ejecuta?

python src/Bloque_C-Boole-Shannon-ComputaciónCuántica/ejercicio_08.py

Corre el caso obligatorio (minterminos {1,3,5,7}, 3 variables) y un segundo ejemplo con 4 variables. Luego pide en modo interactivo el número de variables y la lista de minterminos.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
minterminos {1,3,5,7}, n=3	expresión “C”, misma tabla de verdad	No (caso obligatorio)
minterminos 0-7, n=4	expresión “NOT A”, misma tabla de verdad	No
minterminos [], n=3	expresión “0”	Sí (función siempre falsa)
minterminos 0-7, n=3 (todos)	expresión “1”	Sí (función siempre verdadera)
{1,3,5,7} vs suma directa de 3 literales	simplificada usa 1 solo literal	No (verifica que sí simplifica)
mintermino 8 con n=3 (máx. válido 7)	lanza ValueError	Sí (fuera de rango)
num_variables=5	lanza ValueError	Sí (solo se admite 3 o 4)

5. ¿Qué limitaciones tiene la solución?

La cobertura de implicantes primos no esenciales usa una heurística voraz (igual que el coloreo del ejercicio 6): siempre da una expresión válida y equivalente, pero no está garantizado que sea la más corta posible en todos los casos. Solo admite 3 o 4 variables (A-D), sin minterminos “don’t care”.

Ejercicio 9 — Shannon: medir información en un mensaje

Categoría: boole (información / Shannon)

1. ¿Qué problema resuelve el programa?

Calcula la frecuencia, probabilidad y entropía de Shannon de un texto, y compara la entropía de dos textos explicando cuál tiene mayor incertidumbre y por qué. Como extensión opcional, construye un código de Huffman y compara su longitud promedio contra la entropía.

2. ¿Qué idea matemática usa?

La entropía de Shannon $H = -\sum p_i \log_2(p_i)$ mide la incertidumbre promedio de una fuente de símbolos, en bits por símbolo. $\log_2(1/p_i)$ es la “sorpresa” de ver el símbolo i ; H es el promedio ponderado de esa sorpresa. Un texto repetitivo tiene un símbolo con probabilidad cercana a 1, así que casi no hay sorpresa (H cercano a 0). Un texto con símbolos repartidos de forma pareja es más impredecible (H se acerca a su máximo, $\log_2$ del número de símbolos distintos, en equiprobabilidad exacta). La entropía NO depende del largo del texto, solo de qué tan parejo es el reparto de frecuencias. Huffman construye un árbol combinando siempre los dos símbolos de menor probabilidad; su longitud promedio siempre es $\geq H$ (límite teórico de Shannon para codificación sin pérdida).

3. ¿Cómo se ejecuta?

python src/Bloque_C-Boole-Shannon-ComputaciónCuántica/ejercicio_09.py

Analiza un texto muy repetitivo (“AAAAAAAAAABBBAAAA”) y uno variado (una frase con letras distintas), compara sus entropías, y aplica Huffman sobre el texto variado. Al final hay modo interactivo para ingresar dos textos propios.

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
“AAAAAA” (un solo símbolo)	entropía = 0	Sí (determinista, sin incertidumbre)
“ABABABAB” (2 símbolos equiprobables)	entropía = 1 bit exacto	Sí (caso teórico exacto)
texto repetitivo vs. texto variado	el variado tiene mayor entropía	No (comparación pedida)
“mississippi”	las probabilidades suman 1.0	No (verifica consistencia)
texto = “”	lanza ValueError	Sí (texto vacío)
Huffman sobre texto variado	longitud promedio $\geq$ entropía	No (verifica cota teórica)
Huffman con un solo símbolo (“ZZZZZ”)	código trivial “0”	Sí (un solo símbolo, sin árbol real)

5. ¿Qué limitaciones tiene la solución?

Trata cada carácter como un símbolo independiente (no modela dependencias entre caracteres consecutivos, como en un modelo de Markov); la entropía calculada es la de “orden 0”. El código de Huffman no se usa para comprimir/descomprimir realmente el texto, solo se calcula su longitud promedio para compararla con la entropía.

Ejercicio 10 — Primer simulador cuántico: bits, qubits y mediciones

Categoría: cuantica

1. ¿Qué problema resuelve el programa?

Simula un solo qubit representado como un vector $[\alpha, \beta]$, aplica las compuertas X, Z y H, calcula las probabilidades teóricas de medir 0 o 1 (regla de Born) y simula 1000 mediciones para comparar las frecuencias observadas contra esas probabilidades.

2. ¿Qué idea matemática usa?

Álgebra lineal: el estado es un vector de norma 1, y aplicar una compuerta es multiplicar por una matriz unitaria 2×2 ($|\psi'\rangle = U|\psi\rangle$). X intercambia $|0\rangle$ y $|1\rangle$; Z invierte la fase de $|1\rangle$ sin cambiar probabilidades; H crea superposición (de un estado básico produce 50%/50%). La regla de Born dice que $P(0) = |\alpha|^2$ y $P(1) = |\beta|^2$. Simular una medición es un solo sorteo aleatorio clásico usando esas probabilidades como distribución; por la ley de los grandes números, al repetir 1000 veces la frecuencia observada se acerca a la probabilidad teórica.

3. ¿Cómo se ejecuta?

python src/Bloque_C-Boole-Shannon-ComputaciónCuántica/ejercicio_10.py

Corre los 3 casos obligatorios (X $|0\rangle$, H $|0\rangle$, HH $|0\rangle$), un ejemplo adicional con Z y una combinación de compuertas, y termina con un modo interactivo donde se elige el estado inicial (0 o 1) y una secuencia de compuertas a aplicar (ej. "X Z H").

4. ¿Qué pruebas se hicieron?

Entrada	Salida esperada	¿Caso límite?
X $ 0\rangle$	igual a $ 1\rangle$	No (caso obligatorio)
H $ 0\rangle$	$P(0)=P(1)=0.5$ exacto	No (caso obligatorio)
H(H $ 0\rangle$)	igual a $ 0\rangle$ (con tolerancia numérica)	No (caso obligatorio)
Z $ 1\rangle$	vector $[0, -1]$; P(1) sigue siendo 1.0	Sí (fase cambia, probabilidad no)
1000 mediciones sobre H $ 0\rangle$	frecuencia de 0 cerca de 0.5 (± 0.1)	Sí (variabilidad estadística)
probabilidades de $ 0\rangle$ puro	$P(0)=1.0$, $P(1)=0.0$ exacto	Sí (estado sin superposición)
simular_mediciones con 0 repeticiones	lanza ValueError	Sí (número inválido)

5. ¿Qué limitaciones tiene la solución?

Es una simulación clásica: todo el vector de estado es visible en todo momento, algo que no ocurre en un computador cuántico real (ahí solo se observan los resultados)

0/1 de cada medición, nunca α y β directamente, y cada medición colapsa físicamente el estado). Tampoco hay ruido físico ni decoherencia, que sí afectan a un computador cuántico real. Solo trabaja con 1 qubit (no hay entrelazamiento ni múltiples qubits).
